

Blood Report Platform — Access & Authorization Addendum

Resolving Lab Data-Access and Doctor Consent Gaps in the V1 Architecture — V1.1

Purpose of this document. The V1 product story establishes the core idea correctly: the patient is the central identity, and laboratories and doctors connect to that identity rather than owning it. However, the V1 document itself flags two unresolved gaps at the end — laboratory access to patient identity, and the breadth of doctor consent. This addendum works through both problems in detail, rejects the approaches that create privacy exposure, and specifies the model that should be treated as the source of truth going forward. It should be read as a direct continuation of Section 33 ("The Main Rules of the Platform") and Section 36 ("Gaps") of the original document.

1. Problem Recap
2. Why "Approve the Lab, Then Share Basic Info With All Approved Labs" Doesn't Work
3. Solution — Two Separate Approval Layers for Laboratories
4. Lab Access Flow, Step by Step
5. Why "Permanent Doctor Approval" Creates the Doctor A / Doctor B Problem
6. Solution — Consent as a Record, Not a Flag
7. Doctor Access Flow, Step by Step
8. Data Model Additions
9. Updated End-to-End Sequence
10. Revised Platform Rules (V1.1)
11. Additional Hardening Recommendations
12. Summary of What Changes vs. the Original V1 Document

1. Problem Recap

Two open questions were raised against the V1 design:

Problem 1 — Laboratory access. The patient logs into the platform once. If an admin approves a lab, and every approved lab is then allowed to pull the patient's basic info regardless of whether the patient ever visited that lab, the platform is exposing patient identity data to labs that have no relationship with that patient at all.

Problem 2 — Doctor consent. If patient approval of a doctor is a one-time, permanent switch, then a doctor seen once — six months ago, for one issue — keeps silently receiving every new lab report forever. If the patient later also approves a second doctor, the first doctor's access is not automatically affected, and the patient has no way to see or control who currently holds standing access to their data.

Both problems have the same root cause: **the original design treats "approval" as a single, permanent, all-or-nothing switch**, attached to the wrong entity (the lab or doctor's account) instead of to the specific relationship and moment it should apply to. The fix below is the same shape for both: separate *identity verification* from *data-access consent*, and make consent a first-class, inspectable, revocable record.

2. Why "Approve the Lab, Then Share With All Approved Labs" Doesn't Work

Your proposed model was: admin approves a lab once → every approved lab can then pull any patient's basic info on login, regardless of which lab the patient actually visits. Walking it through:

- It means Lab C — which Rahul has never visited and never will — can pull Rahul's name, DOB, gender, and phone the moment it is "approved," purely because Rahul exists in the system.
- It scales badly: every new lab that joins the platform gains visibility into every existing patient's identity data on day one.
- It directly contradicts the one principle the original document already states correctly in Section 7: "*Lab A does NOT automatically see Lab B's reports... it sees the patient identity, but not another laboratory's medical data.*" Sharing basic info platform-wide undermines that same boundary at the identity layer instead of the report layer.

This is not "not the best idea" — it is the one design in this entire document that would fail a basic privacy audit, because it grants data access based on **lab status** rather than **patient relationship**. The fix is straightforward once the two approvals are separated, below.

3. Solution — Two Separate Approval Layers for Laboratories

Do not merge "is this lab allowed to operate on the platform" with "can this lab see this patient's data." They are different questions, answered by different people, at different times.

Layer	Question it answers	Who approves	What it unlocks
A. Lab Onboarding PLATFORM-LEVEL	Is this a real, licensed laboratory allowed to use the platform at all?	Platform Admin, once, at signup	The lab can log in and <i>search</i> for a Patient ID. Nothing more.
B. Per-Visit Consent PATIENT-LEVEL	Has <i>this</i> patient actually visited <i>this</i> lab, right now, and agreed to a report being attached?	The patient, every visit	The lab may attach/upload a report against that Patient ID.

Layer A is identical to how the original document already treats labs as accounts (Section 20 — "Laboratory Data Ownership"). Layer B is the piece that was

missing. Critically, a lab search on a Patient ID should return only enough information to **confirm identity** — not the patient's full basic profile:

```
Lab searches: PX123456
  ↓
Returns (identity-check only):
  Name:   Rahul S*****
  DOB:    12-05-1995
  Gender: Male
  ↓
Does NOT return: phone number, address, past reports, other labs used
```

4. Lab Access Flow, Step by Step



Why this solves it: it no longer matters how many labs are globally "approved" by the admin — none of them receive any patient data until that specific patient takes a specific action at the point of that specific visit. Lab C, which Rahul has never visited, never even gets a consent prompt, because it never attempts to attach anything to his record. The Patient ID is restored to being a pointer, not a key — matching the requirement already stated in the original document's gap analysis: *"Patient ID must remain an identifier, not proof of permission to upload or inspect data."*

Practical note: for a real clinic visit, an OTP-per-sample-collection may be operationally heavy if the patient is physically present. A reasonable middle ground used by similar systems: the patient generates a short-lived QR/OTP *on their own device* at check-in, the lab operator enters it once to open a 24-48 hour upload window for that one visit, and the window auto-closes. This keeps consent patient-initiated without requiring a live approval tap after every single test.

5. Why "Permanent Doctor Approval" Creates the Doctor A / Doctor B Problem

In the original flow (Section 25), once Rahul authorizes Doctor D001, that doctor gets "all authorized reports" — described with no expiry and no distinction between report at the time of approval versus reports generated afterward. Walking through your scenario:

Month 1: Rahul authorizes Doctor A

→

Doctor A can see all reports, forever, incl. future ones

Month 6: Rahul authorizes Doctor B

→

Doctor B can see all reports, forever, incl. future ones

Month 7: Lab C publishes new report

→

BOTH Doctor A and Doctor B silently receive it

Rahul has no visible list of who currently has access

Rahul has no way to make Doctor A "read-only, historical"

This is precisely the concern the original document raises about itself in its gap analysis: doctor consent is "convenient but too broad," with no defined expiry, no defined revocation path, and no explicit choice about whether future reports are included.

6. Solution — Consent as a Record, Not a Flag

Replace the single doctor-patient "authorized: yes/no" relationship with a **Consent record** — a row in the database representing one specific grant, with its own scope and its own lifecycle. A patient can have many Consent records over time, to many doctors, each independent of the others.

```
Consent {
  id
  patient_id
  doctor_id
  scope
  scope_report_ids      : "all_reports" | "specific_report_ids" | "date_range"
  scope_report_ids      : [ ]           // only if scope = specific_report_ids
  scope_date_range      : { from, to }   // only if scope = date_range
  include_future_reports : true | false  // explicit choice at grant time, default FALSE
  granted_at
  expires_at            : timestamp | "until_revoked"
  status                : active | expired | revoked
  revoked_at
  revoked_by            : patient
}
```

What this single change fixes, directly against your two examples:

Your scenario	How the Consent model resolves it
Rahul approves Doctor A, then later approves Doctor B	Two separate Consent rows. Approving Doctor B does not read, modify, or reference Doctor A's row in any way. Each doctor's access is exactly what its own Consent record says — no more.
Doctor A keeps getting new reports Rahul never meant to keep sharing	At grant time, Rahul explicitly chooses <code>include_future_reports</code> . If he leaves it off (the recommended default), Doctor A only ever sees what existed at the moment of approval — a snapshot, not a subscription. If Rahul does want ongoing access, that's a conscious opt-in, not a silent default.
Rahul wants to stop Doctor A specifically, without touching Doctor B	Patient opens "Manage Doctor Access," sees every active Consent row (one per doctor) with its scope, and revokes Doctor A's row by itself. Status flips to <code>revoked</code> ; any future query from Doctor A against PX123456 returns nothing new. (Reports already downloaded/viewed before revocation cannot be technically un-shown — this is a real-world limit, not something any software design can undo, and is worth stating plainly to users.)
Consent silently lasting forever	<code>expires_at</code> gives every grant a default lifetime (e.g. 6-12 months) unless the patient explicitly selects "until I revoke it." Expired consents fall to <code>status = expired</code> automatically and the doctor loses access without the patient needing to remember to revoke anything.

7. Doctor Access Flow, Step by Step

RAHUL VISITS DOCTOR A (first time)

|

↓

Doctor A requests access (in person, or Rahul initiates from his app)

|

↓

Consent SCREEN SHOWN TO RAHUL:

Doctor: Dr. A

Share: () All reports () Only these reports () Date range

Include future reports? () Yes () No ← default No

Access duration: () 6 months () 12 months () Until I revoke

|

↓

Consent record created → status = active

|

↓

Doctor A dashboard shows exactly the scope Rahul selected

SIX MONTHS LATER — RAHUL VISITS DOCTOR B

|

↓

Same consent screen, independent record created for Doctor B

|
↓

Doctor A's consent record is completely unaffected
(still governed by its own scope/expiry, checked from step above)

RAHUL OPENS "MANAGE DOCTOR ACCESS" AT ANY TIME

|
↓

Sees: Doctor A — All reports, future: No, expires in 2 months [Revoke]
Doctor B — All reports, future: Yes, until revoked [Revoke]

8. Data Model Additions

These additions sit alongside the schema implied by the original document's Section 32 ("Core Database Story") and Section 20 ("Laboratory Data Ownership"). Nothing about the existing Patient / Report / Laboratory structure needs to change — these are new tables plus one new check in the upload path.

Table	Key fields & purpose
lab_onboarding_status	lab_id, status (pending/approved/suspended), approved_by_admin_id, approved_at — Layer A from Section 3. Governs whether a lab can log in and search at all.
visit_consent	id, patient_id, lab_id, method (otp/qr), issued_at, expires_at, status (pending/approved/denied/expired), linked_report_id — Layer B from Section 3. A row must exist with status = approved before a report upload against that patient/lab pair is permitted. This is enforced server-side in the upload handler, not just hidden in the UI — matching the same "Backend Authorization" principle the original document already applies to report ownership in Section 20.
doctor_consent	As specified in Section 6 above — scope, include_future_reports, expires_at, status, revoked_at.
access_log	actor_type (lab/doctor), actor_id, patient_id, action (searched/viewed/uploaded), timestamp — every lookup of a Patient ID is logged, so the "who looked at my data and when" question is always answerable, and the platform gap flagged in the original document's own review ("access logs" under laboratory boundaries) is closed.

9. Updated End-to-End Sequence

This replaces the "Complete Story in One Flow" diagram from Section 30 of the original document with the two new gates (shown as ★) inserted at the points where data actually changes hands.



10. Revised Platform Rules (V1.1)

These extend — and in two cases sharpen — the ten rules already listed in Section 33 of the original document. Unchanged original rules are shown briefly for context; new/modified rules are marked.

#	Rule	Status
1	One patient → one platform Patient ID	Unchanged
2	Each lab owns only its own reports	Unchanged
3	Patient is the central connection point	Unchanged
4	A lab search returns identity-confirmation fields only — never the full profile, and never other labs' data	SHARPENED
5	Only the originating lab can correct its own reports	Unchanged
6	Corrections create versions, never silent overwrites	Unchanged
7	A lab may not attach a report to a patient without a fresh, per-visit patient consent (visit_consent record)	NEW
8	Doctor access is granted as a scoped, time-bound Consent record — never a permanent blanket switch	SHARPENED
9	Inclusion of future reports in a doctor's access is an explicit patient choice, defaulting to No	NEW
10	Every doctor/lab consent is independently revocable without affecting any other party's access	NEW
11	Patient ID is never sufficient by itself to grant medical-data access	Unchanged (restated)
12	Original laboratory PDF remains unchanged	Unchanged
13	V1 does not perform automatic trend analysis	Unchanged
14	Every patient-data access event (search, view, upload) is logged against the actor	NEW

11. Additional Hardening Recommendations

Beyond the two problems you raised, these follow directly from the gap analysis at the end of the original document and are worth deciding on now, since they affect the same schema:

11.1 — Preventing duplicate patient accounts

The original document flags that Rahul could register twice with different phone numbers, or a lab could attach a report to the wrong "Rahul." Recommend: require one verified, unique field at registration (phone number OR a government ID number, verified via OTP/ID-check), and run a fuzzy duplicate check (name + DOB + gender) at signup that flags likely duplicates for admin review rather than silently creating a second Patient ID.

11.2 — Strengthening the identity check at upload time

Name + DOB + gender alone is thin enough that a typo could attach an accurate report to the wrong person — the original document calls this a "critical failure." Recommend adding one more check before upload is allowed: last four digits of registered phone, or a QR code the patient shows at check-in that encodes their Patient ID directly (removing manual entry/typo risk entirely).

11.3 — QR onboarding flow (Section 28 of the original) should route through the same Gate 2

When a doctor generates a QR for a brand-new patient, the scan should not silently grant blanket access. It should register the patient (creating PX123456) and then immediately present the same scoped consent screen from Section 7 above — so QR onboarding and manual doctor-access requests go through one single consent mechanism, not two different ones.

12. Summary of What Changes vs. the Original V1 Document

The core story does not change. Patient-as-central-identity, lab report ownership, structured processing, and the four-layer architecture all remain exactly as designed. What changes is narrow and specific:

- Lab approval by the admin (Layer A) no longer implies any patient-data visibility by itself.
- A new, per-visit, patient-approved gate (Layer B / `visit_consent`) is required before any report can be attached to a patient by any lab.
- Doctor authorization becomes a scoped, expiring, independently revocable `doctor_consent` record instead of a permanent on/off relationship — solving both the Doctor A/Doctor B independence problem and the "silent future reports forever" problem in one data model change.
- All patient-data access is logged.

These are additive changes to the schema and to the upload/authorization code paths — they do not require re-architecting Layers 1–4 as already defined in Section 31 of the original document.

This addendum should be read alongside, and does not replace, *BLOOD REPORT — Connected Healthcare Platform, V1*. Where this document specifies a rule, flow, or schema element not present in V1, treat this addendum as the current source of truth for that item.